

[Sign in](#)

[VisveswarranH](#) / [Implementation-of-filter](#) Public

forked from [swedha333/Implementation-of-filter](#)

GPL-3.0 license

0 stars 426 forks Branches Tags Activity

Star

Notifications

Code Pull requests Actions Projects Security Insights

main



Go to file

Code

This branch is [2 commits ahead of](#) [swedha333/Implementation-of-filter:main](#).



VisveswarranH Add files via upload

b3f4817 · now



Exp5 (1).ipynb

Add files via upload

now



LICENSE

Initial commit

last year



README.md

Update README.md

32 minutes ago



SHARPENING FILTERS.py

check

last year



SMOOTHING FILTERS.py

SMOOTHING FILTERS.py

7 months ago



README



License



Implementation-of-filter

Aim:

To implement filters for smoothing and sharpening the images in the spatial domain.

Software Required:

Algorithm:

Step 1 :

Import the required libraries.

Step 3 :

Read the image by using imread().

Step 3 :

Apply the required filters for the image separately.

Step 4 :

Plot the original and filtered image by using matplotlib.pyplot.

Step 5 :

End the program.

Program :

Developed By : VISVESWARRAN H

Register Number : 212224110063

1. Smoothing Filters

i) Using Averaging Filter

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('Fish.jpg', cv2.IMREAD_GRAYSCALE)
salt_prob = 0.05
pepper_prob = 0.05
```



```

noisy_image = np.copy(image)

num_salt = np.ceil(salt_prob * image.size)
coords_salt = [np.random.randint(0, i, int(num_salt)) for i in image.shape]
noisy_image[tuple(coords_salt)] = 255

num_pepper = np.ceil(pepper_prob * image.size)
coords_pepper = [np.random.randint(0, i, int(num_pepper)) for i in image.sh
noisy_image[tuple(coords_pepper)] = 0

filtered_image = np.zeros_like(noisy_image)
height, width = noisy_image.shape

for i in range(1, height - 1):
    for j in range(1, width - 1):
        neighborhood = noisy_image[i - 1:i + 2, j - 1:j + 2]

        filtered_value = np.mean(neighborhood)

        filtered_image[i, j] = filtered_value

plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(noisy_image, cmap='gray')
plt.title('Noisy Image (Salt-and-Pepper)')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(filtered_image, cmap='gray')
plt.title('Filtered Image (Manual Box Filter 3x3)')
plt.axis('off')

plt.tight_layout()
plt.show()

```

ii) Using Weighted Averaging Filter

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('Fish.jpg', cv2.IMREAD_GRAYSCALE)

```



```

salt_prob = 0.05
pepper_prob = 0.05

noisy_image = np.copy(image)

num_salt = np.ceil(salt_prob * image.size)
coords_salt = [np.random.randint(0, i, int(num_salt)) for i in image.shape]
noisy_image[tuple(coords_salt)] = 255

num_pepper = np.ceil(pepper_prob * image.size)
coords_pepper = [np.random.randint(0, i, int(num_pepper)) for i in image.sh
noisy_image[tuple(coords_pepper)] = 0

kernel = np.array([[1, 2, 1],
                    [2, 4, 2],
                    [1, 2, 1]]) / 16.0

image_height, image_width = noisy_image.shape
kernel_size = kernel.shape[0]
pad = kernel_size // 2

padded_image = np.pad(noisy_image, pad, mode='constant', constant_values=0)

filtered_image = np.zeros_like(noisy_image)

for i in range(pad, image_height + pad):
    for j in range(pad, image_width + pad):
        roi = padded_image[i - pad:i + pad + 1, j - pad:j + pad + 1]

        filtered_value = np.sum(roi * kernel)

        filtered_image[i - pad, j - pad] = np.clip(filtered_value, 0, 255)

plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(noisy_image, cmap='gray')
plt.title('Noisy Image (Salt-and-Pepper)')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(filtered_image, cmap='gray')
plt.title('Filtered Image (Manual Weighted Avg)')
plt.axis('off')

```

```
plt.tight_layout()
plt.show()
```

iii) Using Gaussian Filter

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('Fish.jpg', cv2.IMREAD_GRAYSCALE)

blurred_image = cv2.GaussianBlur(image, (3, 3), 0)

laplacian_kernel = np.array([[0, -1, 0],
                             [-1, 4, -1],
                             [0, -1, 0]])

image_height, image_width = blurred_image.shape
kernel_height, kernel_width = laplacian_kernel.shape
pad_height = kernel_height // 2
pad_width = kernel_width // 2

padded_image = np.pad(blurred_image, ((pad_height, pad_height), (pad_width,
laplacian_image = np.zeros_like(blurred_image)

for i in range(image_height):
    for j in range(image_width):
        region = padded_image[i:i + kernel_height, j:j + kernel_width]
        laplacian_value = np.sum(region * laplacian_kernel)
        laplacian_image[i, j] = laplacian_value

laplacian_image = np.clip(laplacian_image, 0, 255).astype(np.uint8)

sharpened_image = cv2.add(image, laplacian_image)

plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(laplacian_image, cmap='gray')
plt.title('Laplacian Filtered Image')
plt.axis('off')

plt.subplot(1, 3, 3)
```

```
plt.imshow(sharpened_image, cmap='gray')
plt.title('Sharpened Image')
plt.axis('off')

plt.tight_layout()
plt.show()
```

iv) Using Median Filter

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('Fish.jpg', cv2.IMREAD_GRAYSCALE)

salt_prob = 0.05
pepper_prob = 0.05

noisy_image = np.copy(image)

num_salt = np.ceil(salt_prob * image.size)
coords_salt = [np.random.randint(0, i, int(num_salt)) for i in image.shape]
noisy_image[tuple(coords_salt)] = 255

num_pepper = np.ceil(pepper_prob * image.size)
coords_pepper = [np.random.randint(0, i, int(num_pepper)) for i in image.sh
noisy_image[tuple(coords_pepper)] = 0

filtered_image = np.zeros_like(noisy_image) # Create an empty output image

height, width = noisy_image.shape

for i in range(1, height - 1):
    for j in range(1, width - 1):
        neighborhood = noisy_image[i - 1:i + 2, j - 1:j + 2]

        median_value = np.median(neighborhood)

        filtered_image[i, j] = median_value

plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(noisy_image, cmap='gray')
```

```
plt.title('Noisy Image (Salt-and-Pepper)')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(filtered_image, cmap='gray')
plt.title('Filtered Image (Manual Median Filter)')
plt.axis('off')

plt.tight_layout()
plt.show()
```

2. Sharpening Filters

i) Using Laplacian Linear Kernel

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('Fish.jpg', cv2.IMREAD_GRAYSCALE)

blurred_image = cv2.GaussianBlur(image, (3, 3), 0)

laplacian_kernel = np.array([[0, -1, 0],
                             [-1, 4, -1],
                             [0, -1, 0]])

image_height, image_width = blurred_image.shape
kernel_height, kernel_width = laplacian_kernel.shape

pad_height = kernel_height // 2
pad_width = kernel_width // 2

padded_image = np.pad(blurred_image, ((pad_height, pad_height), (pad_width,
laplacian_image = np.zeros_like(blurred_image)

for i in range(image_height):
    for j in range(image_width):
        region = padded_image[i:i + kernel_height, j:j + kernel_width]
        laplacian_value = np.sum(region * laplacian_kernel)
        laplacian_image[i, j] = laplacian_value

laplacian_image = np.clip(laplacian_image, 0, 255).astype(np.uint8)

sharpened_image = cv2.add(image, laplacian_image)

plt.figure(figsize=(12, 4))
```

```
plt.subplot(1, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(laplacian_image, cmap='gray')
plt.title('Laplacian Filtered Image')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(sharpened_image, cmap='gray')
plt.title('Sharpened Image')
plt.axis('off')

plt.tight_layout()
plt.show()
```

ii) Using Laplacian Operator

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('Fish.jpg', cv2.IMREAD_GRAYSCALE)

blurred_image = cv2.GaussianBlur(image, (3, 3), 0)

laplacian_image = cv2.Laplacian(blurred_image, cv2.CV_64F, ksize=3)

laplacian_image = np.absolute(laplacian_image)
laplacian_image = np.clip(laplacian_image, 0, 255).astype(np.uint8)

sharpened_image = cv2.add(image, laplacian_image)

plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(laplacian_image, cmap='gray')
plt.title('Laplacian Filtered Image')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(sharpened_image, cmap='gray')
```

```
plt.title('Sharpened Image')
plt.axis('off')

plt.tight_layout()
plt.show()
```

OUTPUT:

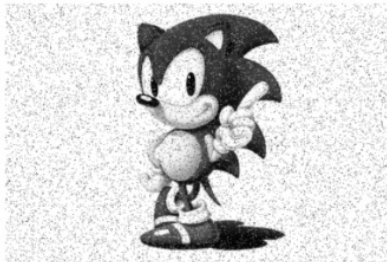
1. Smoothing Filters

i) Using Averaging Filter

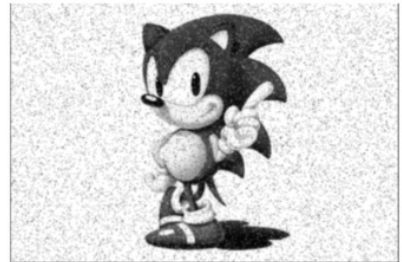
Original Image



Noisy Image (Salt-and-Pepper)



Filtered Image (Manual Box Filter 3x3)

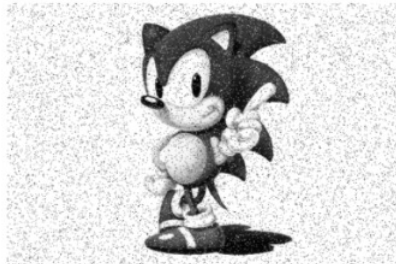


ii) Using Weighted Averaging Filter

Original Image



Noisy Image (Salt-and-Pepper)



Filtered Image (Manual Weighted Avg)

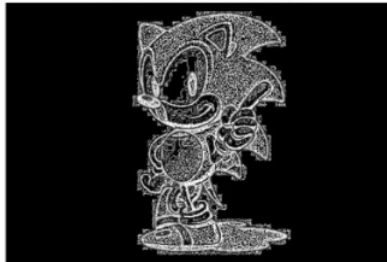


iii) Using Gaussian Filter

Original Image



Laplacian Filtered Image



Sharpened Image



iv) Using Median Filter

Original Image

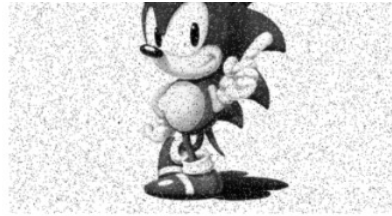


Noisy Image (Salt-and-Pepper)



Filtered Image (Manual Median Filter)





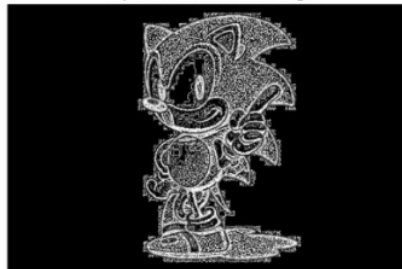
2. Sharpening Filters

i) Using Laplacian Kernel

Original Image



Laplacian Filtered Image



Sharpened Image



ii) Using Laplacian Operator

Original Image



Laplacian Filtered Image



Sharpened Image



Result:

Thus the filters are designed for smoothing and sharpening the images in the spatial domain.

Releases

No releases published

Packages

No packages published

Languages



[Terms](#) [Privacy](#) [Security](#) [Status](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)



© 2025 GitHub, Inc.